

Coordinators and Subagents

What a subagent actually is, why context isolation is the real reason to spawn one, and how delegation fails quietly.

A subagent is not a helper. It is a **separate conversation** with its own context window.

Four reasons to spawn one

- ◆ **Context isolation.** Fifty files read inside the subagent, one summary back. The parent's context stays clean.
- ◆ **Parallelism.** Independent subagents run concurrently, so the work takes as long as the slowest one.
- ◆ **Specialised instructions.** A migration expert's prompt that would be noise in the main agent.
- ◆ **Tool restriction.** A reviewer that can read but can never write.

ON THE EXAM

Isolation is the one that matters most. The others are conveniences you could achieve other ways.

What happens when the coordinator delegates

01

Coordinator calls

The Agent tool, with a subagent type and a prompt.

02

Fresh context

The subagent starts a new conversation. It sees the prompt, not your history.

03

It works

Its own tool calls, its own results, all inside its own window.

04

One message back

The final message returns as the tool result. Nothing else does.

ON THE EXAM

The parent never sees the subagent's intermediate tool calls. That is the feature, not a limitation.

What a subagent does not inherit

Fresh context means fresh. This list is longer than people expect.

- ◆ **Your conversation history.** It has no idea what the user asked or what has been decided.
- ◆ **Files already read in the main thread.** If it needs them, it reads them again.
- ◆ **Findings from a sibling subagent.** There is no shared memory between them.
- ◆ **The only channel is the prompt you pass in the Agent call.** If it is not in there, it does not exist.

ON THE EXAM

"Pass findings in the prompt" is the single most repeated correct answer in this part of the syllabus.

Fresh subagent or fork?

FRESH

Starts from nothing

- Empty context plus the prompt you pass
- Maximum isolation, minimum inheritance
- You must restate anything it needs
- The default, and usually what you want

FORK

Starts from the parent's state

- Inherits the parent conversation at that point
- No need to restate context it already has
- Costs the parent's context on every fork
- For exploring two branches from one position

THE TIEBREAKER

Either way, only the final message returns. Fork changes what goes in, never what comes out.

Delegation is not **sharing** work. It is **describing** work well enough that someone with no context can do it.

What belongs in a delegation prompt

- ◆ **The objective**, stated narrowly enough that two subagents cannot both think it is theirs.
- ◆ **The output format** you need back, since the final message is all you get.
- ◆ **Context and findings** from earlier work, restated in full. It cannot look them up.
- ◆ **Boundaries**. What is explicitly out of scope, and which sources or paths to use.
- ◆ **Which tools to prefer**, when the subagent has several that overlap.

ON THE EXAM

Duplicated work and coverage gaps are both symptoms of vague briefs, not of too few subagents.

Hub and spoke

01

Coordinator

Holds the goal, the plan and every result.

02

Delegates out

One prompt per subagent, written from what it knows.

03

Results return

Each final message comes back to the coordinator alone.

04

Coordinator synthesises

It is the only place where findings ever meet.

ON THE EXAM

Subagents never message each other. If B needs what A found, the coordinator puts it in B's prompt.

How parallel delegation actually happens

- ◆ **Several Agent calls in one assistant response** run concurrently. That is the whole mechanism.
- ◆ **One call, wait for the result, then another** is sequential, whatever the prompt says.
- ◆ **Telling the model to "work in parallel"** changes nothing by itself. Concurrency is a property of the response shape.
- ◆ **Agent must be in allowedTools**, or the coordinator reasons about delegating and never does it.

ON THE EXAM

You define the capability. Claude decides the scheduling. Those are different things, and questions exploit the gap.

Four ways delegation fails quietly

SYMPTOM	CAUSE
Subagents cover the same ground	Objectives overlap. The briefs were not narrow enough.
An obvious topic is missing entirely	Each subagent assumed another had it. Nobody was told it was theirs.
Subagents run one after another	One Agent call per response instead of several in one.
It talks about delegating but never does	Agent is missing from allowedTools.

ON THE EXAM

None of these throws an error. All of them look like the system working, just badly.

Gap-driven re-delegation

The coordinator's second job, and the one people forget it has.

- ◆ **Results come back.** The coordinator reads them and asks what is still missing.
- ◆ **It spawns further subagents for the gaps,** with prompts written from what the first round found.
- ◆ **This is why the subtask list cannot be fixed at author time.** Round two depends on round one.
- ◆ **Without it, you have parallelisation.** Fire once, aggregate, hope the split was right.

ON THE EXAM

"Spawn more subagents based on what came back" is the orchestrator-worker pattern doing its actual job.

Spawn a subagent, or just use more context?

DELEGATE

The work needs its own window

- Exploration generates a lot of throwaway output
- The parent is already degrading
- The task needs different tools or restrictions
- Several pieces are genuinely independent

DO IT INLINE

The parent already has it

- The answer is in context right now
- The work is small and generates little
- You would be re-reading what you have
- A follow-up question about existing findings

THE TIEBREAKER

Spawning a subagent to re-read what the coordinator is already holding is pure overhead. Slower and more expensive.

What a multi-agent system costs

- ◆ **Tokens multiply.** Anthropic measured their own research system at roughly fifteen times the tokens of a chat interaction.
- ◆ **Every subagent pays a context loading cost** before it does anything useful.
- ◆ **Parallelism buys wall-clock, never spend.** Six concurrent subagents cost the same as six sequential ones.
- ◆ **Worth it when the task value is high and the work genuinely parallelises.** Not otherwise.

ON THE EXAM

"Reduce cost" and "reduce latency" pull in different directions here. Read which one the question asked for.

Two configuration details worth marks

- ◆ **A subagent inherits the lead's model unless you set one.** A cheap triage task silently runs on your most expensive model.
- ◆ **Tool restriction is a real control**, not a hint. A subagent given only read tools cannot write, however it is prompted.
- ◆ **maxTurns can be set per subagent.** Hitting it returns partial output, which the parent must handle.
- ◆ **Claude chooses which subagent to invoke from its description field.** Vague descriptions produce wrong delegation.

ON THE EXAM

The description is how Claude routes work. Write it as "use this when...", not as a job title.

Currency check: what has moved

TOPIC	EXAM ANSWER	CURRENT REALITY
The delegation tool	Task	Renamed Agent in Claude Code 2.1.63. Task still works as an alias.
Nesting	One level. Subagents cannot spawn subagents.	Docs describe nesting up to five levels; reports differ on which types get it.
Sibling communication	None. Coordinator is the only channel.	Unchanged. Still the rule.

ON THE EXAM

Answer Task and one level on the exam. Know the rest exists so the docs do not confuse you.

Everything a subagent knows, you wrote.
Everything you learn from it fits in **one**
message.

Module 3 in five lines

- ◆ A subagent is a separate conversation with its own context. Isolation is the main reason to spawn one.
- ◆ It inherits nothing. The prompt is the only channel in; the final message is the only channel out.
- ◆ Hub and spoke. Subagents never talk to each other; the coordinator carries findings between them.
- ◆ Concurrency comes from several Agent calls in one response, not from asking nicely.
- ◆ Subagents buy wall-clock time and context headroom. They never buy cost.

ON THE EXAM

Six worked scenarios next. Two of them look like delegation problems and are not.

A research system has a coordinator that spawns four specialised subagents: a web search agent, an academic paper agent, a data analysis agent and a report generator.

The report generator consistently produces reports that omit findings the data analysis agent uncovered. Logs confirm the data analysis subagent completed successfully and returned a detailed result to the coordinator. The report generator is spawned immediately afterwards with the prompt "Generate a comprehensive report on the research question."

What is the root cause?

- ☐ A The report generator needs a longer maxTurns so it has time to gather the analysis
- ☐ B The data analysis subagent should message the report generator directly with its findings
- ☐ C The coordinator is not including the analysis findings in the report generator's prompt
- ☐ D The report generator's model is too small to handle comprehensive synthesis

◆ PAUSE AND DECIDE

What is the root cause?

- A

 Longer maxTurns
More turns to search a context that will never contain the findings. It cannot gather what it was never given.
- B

 Direct subagent messaging
Subagents have no channel to each other. This describes a mechanism that does not exist.
- C

 Findings not in the prompt
The quoted prompt contains no findings, and the prompt is the only channel in. Nothing else could explain it.
- D

 Model too small
Omission of data it never received is not a capability problem. A larger model omits it equally.

THE SIGNAL

The prompt is quoted in full in the stem. That is deliberate. Read it and notice what is not there.

A coordinator handles research requests that need three independent lines of enquiry. Each is genuinely separable and none depends on another's output.

The system prompt says: "Delegate independent research tasks to subagents and run them in parallel wherever possible." Traces show the coordinator issuing one Agent call, waiting for its result, issuing the next, and so on. Total time is roughly three times what the team expected.

allowedTools includes Agent, and each subagent completes correctly.

What change makes them run concurrently?

- ☐ A Add stronger wording to the system prompt, stating that parallel execution is required rather than optional
- ☐ B Structure the coordinator so all three Agent calls are emitted in a single assistant response
- ☐ C Increase maxTurns so the coordinator has room to issue all three calls
- ☐ D Merge the three lines of enquiry into one subagent with a broader brief

◆ PAUSE AND DECIDE

What change makes them run concurrently?

- A

 Stronger prompt wording
The instruction is already there and already being followed loosely. Emphasis does not change response shape.
- B

 Three calls in one response
Concurrency is a property of the message. Multiple Agent calls in one assistant turn execute together.
- C

 Raise maxTurns
It issued all three calls. Turns were never the constraint.
- D

 Merge into one subagent
Removes the parallelism problem by removing the parallelism. Also destroys the context isolation.

THE SIGNAL

"The system prompt already instructs it" is the eliminator. When an instruction exists and is not working, stop writing instructions.

A codebase exploration agent uses subagents to investigate different modules. During a large investigation the coordinator's own context fills up and its later decisions become noticeably worse.

Review shows the coordinator passes each subagent a full copy of everything found so far, and each subagent returns a complete transcript of its exploration, including every file it read, which the coordinator keeps.

What is the most effective change?

- A Have subagents return only their conclusions, and pass each new subagent only the findings relevant to its task
- B Increase the coordinator's model to one with a larger context window
- C Reduce the number of subagents so less material comes back
- D Have each subagent write its transcript to a file and give the coordinator the paths

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 Conclusions out, relevant findings in
Fixes both leaks. Isolation only works if the boundary is narrow in both directions.
- B

 Bigger context window
Buys time and changes nothing structural. The same growth rate fills a bigger window.
- C

 Fewer subagents
Less investigation, therefore less context. Solves the symptom by doing less work.
- D

 Transcripts to files
A reasonable second step, and it fixes only the return leak. The coordinator is still pushing everything downwards.

THE SIGNAL

This system spawned subagents and then defeated the isolation at both ends. Both directions have to be narrow.

A coordinator delegates to five subagents, each investigating one aspect of a market research question. The team's brief to each is a single line: "Research this topic thoroughly and report what you find."

The final reports have two recurring problems. Three subagents independently researched the same regulatory background. And a topic the client specifically asked about appears in no subagent's output at all.

What is the most effective fix?

- ☐ A Add a sixth subagent dedicated to the missing topic
- ☐ B Give each subagent a narrow objective, explicit out-of-scope boundaries, and the required output format
- ☐ C Have the coordinator deduplicate the overlapping sections during synthesis
- ☐ D Run the five subagents sequentially so each can see what the previous one covered

◆ PAUSE AND DECIDE

What is the most effective fix?

- A

 A sixth subagent
Patches this gap and not the next one. The briefs still allow overlap and still allow holes.
- C

 Deduplicate at synthesis
Hides the wasted work and does nothing about the missing topic. Half a fix at best.
- B

 Narrow objectives and boundaries
Overlap and gaps are the same defect: nobody was told precisely what was theirs.
- D

 Run them sequentially
Subagents cannot see each other's work regardless of order. Sequential just makes it slower.

THE SIGNAL

Duplicated coverage and missing coverage are one problem with two faces. Only an answer addressing both is the fix.

A support engineering team runs an agent that investigates incidents. It spawns subagents to read logs, query metrics and search the runbook repository. This works well.

The team wants to reduce cost. Their proposal is to run the three subagents concurrently rather than sequentially, which they expect to cut the bill by roughly two thirds.

Latency is not currently a problem. Incidents are investigated in the background and results are reviewed the next morning.

What should you tell them?

- ☐ A Concurrency will not reduce cost. Consider a cheaper model for the subagents, or narrower briefs that read less
- ☐ B The estimate is roughly right, since concurrent execution shares context loading across the three subagents
- ☐ C Concurrency reduces cost only if the subagents use the same tools
- ☐ D Merge the three subagents into one to eliminate two context loads

◆ PAUSE AND DECIDE

What should you tell them?

- A

 Concurrency is free of cost benefit
Correct on both halves. Tokens are unchanged by scheduling, and it names two levers that do reduce spend.
- B

 Shared context loading
Invented mechanism. Each subagent loads its own context whether it runs now or later.
- C

 Only with shared tools
Tool overlap has no bearing on billing. Plausible-sounding and entirely made up.
- D

 Merge into one
Saves two context loads and gives up isolation. The stem never said the design was wrong.

THE SIGNAL

"Latency is not a problem" is the planted detail. It tells you the one thing concurrency does buy is worthless here.

A code review agent is being extended. The team wants a subagent that reviews changes for security issues and reports what it finds, but that must never be able to modify files, regardless of how it is prompted or what a reviewer asks it to do.

One engineer suggests a system prompt for the subagent beginning "You are a read-only reviewer. Never edit, write or delete files under any circumstances."

What is the most reliable way to enforce this?

- ☐ A Use the suggested system prompt, and add the constraint to CLAUDE.md as reinforcement
- ☐ B Define the subagent with only read tools, so write tools are not in its toolset at all
- ☐ C Have the coordinator inspect the subagent's output and reject any file modifications
- ☐ D Run the subagent with maxTurns set to 1 so it has no opportunity to write

◆ PAUSE AND DECIDE

What is the most reliable way to enforce this?

- A

 System prompt plus CLAUDE.md
Both are context. Context persuades and cannot guarantee. "Regardless of how it is prompted" rules this out by definition.
- B

 Read tools only
A tool it does not have is a tool it cannot call. The restriction holds under any prompt.
- C

 Coordinator inspects output
Detection after the fact. The write already happened, and the coordinator only sees the final message.
- D

 maxTurns = 1
One turn is enough to call a write tool. It also cripples a reviewer that needs to read several files.

THE SIGNAL

"Regardless of how it is prompted" is the qualifier from module 0. It eliminates every prompt-shaped answer before you read them.